

408 统考 · 考纲全覆盖

数据结构 专业名词手册

按 408 统考考纲收录 **190+** 核心专业名词：

线性表、树、图、查找与排序全家族；

附操作复杂度速查、排序对比总表与逻辑结构分类图。

2027 考研 · 408 统考复习资料

DATA STRUCTURES · TERMINOLOGY HANDBOOK

2026 年 9 月 · SEPTEMBER 2026

目 录

第1章 数据结构与算法基础	2
1.1 基本概念与三要素	2
1.2 算法与复杂度	3
第2章 线性表、栈、队列和串	3
2.1 线性表与两种存储	3
2.2 栈与队列	4
2.3 串与数组	5
第3章 树与二叉树	5
3.1 树与二叉树基本性质	5
3.2 遍历与线索二叉树	6
3.3 树的存储、转换与哈夫曼树	6
第4章 图	7
4.1 图的基本概念与存储	7
4.2 遍历与经典算法	8
第5章 查找	9
5.1 线性与树形查找	9
5.2 B 树、B+ 树与散列表	9
第6章 排序	10
6.1 排序家族	10
6.2 排序算法对比总表（必背）	12
第7章 跨章关系与易混辨析	12
7.1 操作复杂度速查	12
7.2 易混名词辨析	13

第1章 数据结构与算法基础

本章是全书的坐标系：数据结构三要素（逻辑结构、存储结构、数据运算）界定了每一个后续名词的归属，算法与复杂度则是评价一切操作的标尺。看懂第1张图（图1-1），全书的名词网就有了骨架。

1.1 基本概念与三要素

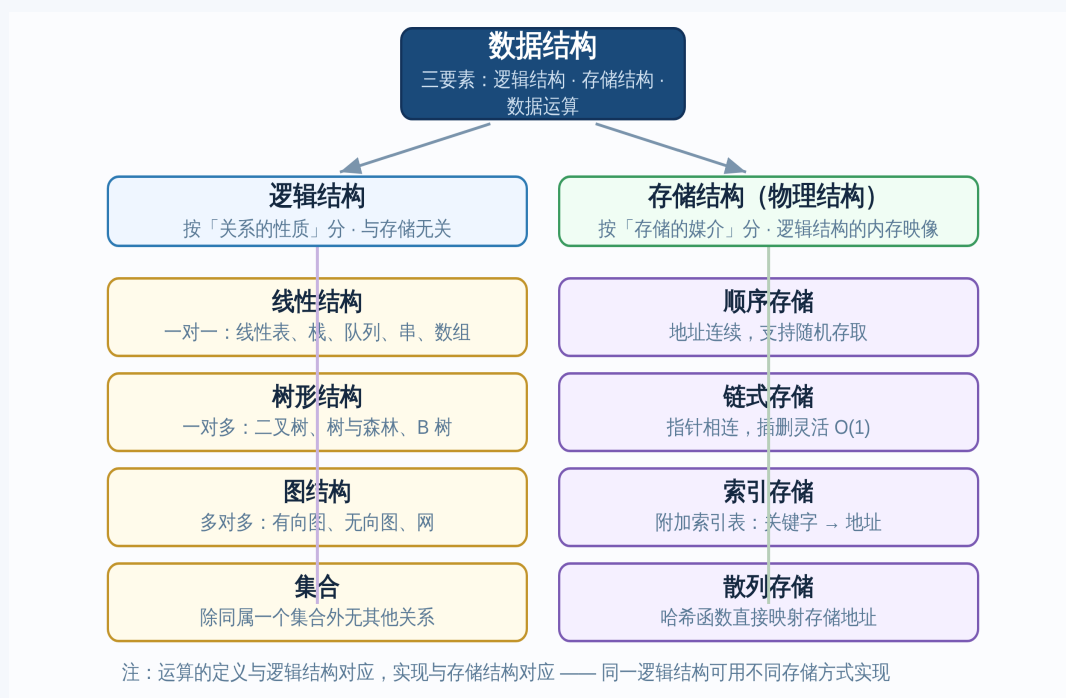


图1-1 数据结构总体分类：逻辑结构决定「关系」，存储结构决定「怎么放」

名词	英文/缩写	含义与要点
数据	Data	能被计算机识别、存储和处理的符号集合，是计算机加工处理的对象。
数据元素 / 数据项	Data Element / Item	数据的基本单位，通常作为整体处理；一个元素可由若干数据项（最小、不可再分的单位，如学号）组成。结点、记录是其别称。
数据对象	Data Object	具有相同性质的数据元素的集合，是数据的一个子集。
数据类型 / 抽象数据类型	ADT	数据类型=值集合+操作集合；抽象数据类型用三元组（数据对象、数据关系、基本操作）逻辑描述，与实现分离，体现封装思想。
数据结构三要素	—	逻辑结构：元素间逻辑关系，与存储无关；存储结构（物理结构）：逻辑结构在内存中的映像；数据运算：运算的定义针对逻辑结构、实现针对存储结构。
逻辑结构	Logical Structure	分为线性结构（线性表、栈、队列、串、数组、广义表）与非线性结构（树、图、集合）。线性结构中有且仅有一个开始结点和一个终端结点，其余结点仅有一个直接前驱和一个直接后继。

名词	英文/缩写	含义与要点
存储结构	Storage Structure	顺序存储：地址连续、可随机存取、插删需移动元素；链式存储：指针相连、插删 $O(1)$ 、不能随机存取且指针占额外空间；索引存储：索引表（关键字+地址）加速查找；散列存储：由哈希函数直接映射地址。逻辑结构独立于存储结构，同一种逻辑结构可采用不同存储实现。

表1-1 基本概念名词

1.2 算法与复杂度

名词	英文/缩写	含义与要点
算法	Algorithm	对特定问题求解步骤的描述。五个特性：有穷性（有限步内结束）、确定性、可行性、输入、输出。好算法目标：正确性、可读性、健壮性、高效率（时间与空间）。
时间复杂度	$T(n)$	$T(n)=O(f(n))$ ：问题规模 n 增长时执行次数的数量级，只保留最高阶项且系数取 1。常见阶由低到高： $O(1)<O(\log_2n)<O(n)<O(n\log_2n)<O(n^2)<O(n^3)<O(2^n)<O(n!)$ 。「常对幂指阶」速记。
空间复杂度	$S(n)$	算法所需辅助空间关于 n 的数量级。原地工作指 $S(n)=O(1)$ ；递归算法的栈深度计入空间复杂度（如快排平均 $O(\log_2n)$ 、最坏 $O(n)$ ）。
大 O 与大 Ω 、 Θ	Big-O / Ω / Θ	O 为渐近上界， Ω 为渐近下界， Θ 为同阶紧确界。教材与考题通常用大 O 表示最坏情况下的渐近上界；分析时以最坏情况为准，有时参考平均情况（如快排）。

表1-2 算法名词

第2章 线性表、栈、队列和串

线性结构是出现频率最高的考点群。主线：先掌握线性表两种基本存储（顺序、链式）的性能差异，再理解受限于「只在一端操作」的栈与「一端进另一端出」的队列，最后是内容为字符的特殊线性表——串及其模式匹配算法 KMP。

2.1 线性表与两种存储

名词	英文/缩写	含义与要点
线性表	Linear List	n ($n \geq 0$) 个数据元素的有限序列。位序从 1 计，存储下标从 0 计。
顺序表	Sequential List	用一段地址连续的存储单元依次存放。特点：逻辑相邻物理也相邻；可随机存取（按位 $O(1)$ ）；按值查找 $O(n)$ ；插入删除平均移动 $n/2$ 个元素， $O(n)$ 。存储密度=1。

名词	英文/缩写	含义与要点
单链表	Linked List	每个结点含数据域与指向后继的指针域。特点：不能随机存取，按位查找 $O(n)$ ；已知前驱时删除 $O(1)$ 。头指针始终指向链表第一个结点（含头结点时即头结点），是存取入口、必不可少；头结点为统一删除逻辑而设，可无。
双链表	Doubly Linked List	结点增加前驱指针 prior，可 $O(1)$ 找到前驱，删除需同时维护 4 个指针域，注意先连后断防断链。
循环链表	Circular List	尾结点指针回指（循环单链表尾结点 next 指头结点；循环双链表头结点 prior 指尾结点）。设尾指针 rear 时，rear->next->next 即首元结点，头部尾部操作均 $O(1)$ 。
静态链表	Static Linked List	用数组模拟链表：游标（下标）代替指针。容量固定，适合无指针的语言环境。
顺序表与链表对比	—	存取方式：顺序可随机存取、链式只能顺序存取；删除：链表改指针更优；空间：顺序需预估容量可能溢出、链式有指针开销但动态分配。选择依据：表长变化大、频繁删除选链式；频繁按位存取、规模稳定选顺序。

表2-1 线性表名词

2.2 栈与队列

名词	英文/缩写	含义与要点
栈	Stack	只在栈顶（表尾）插入删除的线性表，后进先出 LIFO。基本操作 push/pop/getTop。n 个不同元素入栈，出栈序列数 = 卡特兰数 $C(2n,n)/(n+1)$ 。
顺序栈 / 共享栈	—	顺序栈用 top 指示栈顶（约定指向栈顶元素或栈顶空位，注意题目口径）；两栈共享一个数组、栈顶相向而设即为共享栈，栈满条件 $top1+1=top2$ 。
链栈	Linked Stack	用链式存储实现，通常不设头结点，栈顶即表头，插入删除 $O(1)$ ，无栈满问题。
队列	Queue	只允许队尾插入（入队）、队头删除（出队）的线性表，先进先出 FIFO。
循环队列	Circular Queue	用取模运算使数组首尾相接。判满三种方法：牺牲一个单元（ $(rear+1)\%MaxSize==front$ ）；增设 size 变量；增设 tag 标志记录最近操作是入队还是出队。队空： $front==rear$ 。
双端队列	Deque	两端均可插入删除。受限双端队列：一端只删一端只插（输入受限/输出受限）。判断输出序列合法性是常考题。
栈的应用	—	括号匹配、表达式求值（中缀→后缀：操作数直接输出、运算符按优先级进出栈；后缀求值用一个栈）、递归（函数调用栈保存返回地址、参数、局部变量）、进制转换。
队列的应用	—	BFS 遍历图、操作系统就绪队列/缓冲区、页面置换算法实现。

表2-2 栈与队列名词

2.3 串与数组

名词	英文/缩写	含义与要点
串	String	内容受限为字符的线性表。定长顺序存储、堆分配存储、块链存储三种实现。
模式匹配 / BF 算法	Brute-Force	主串长 n 、模式串长 m 。BF 暴力匹配最坏 $O(nm)$ ：失配后主串指针回溯。
KMP 算法	Knuth-Morris-Pratt	利用 next 数组实现主串指针不回溯的匹配，时间 $O(n+m)$ 。next[j]=最长相等前后缀长度+1 (1-based)；手算 next 是高频考点。nextval 为 next 的优化，直接跳过必然失配的比较。
特殊矩阵压缩	—	对称矩阵：只存下三角，共 $n(n+1)/2$ 个元素，sa[i][j] 与 sa[j][i] 映射同一单元；三对角矩阵按带压缩，一维下标与 $k=2i+j-3$ 类公式互算；上/下三角矩阵同理。地址计算题先画行列下标起点 (0 或 1)。
稀疏矩阵	Sparse Matrix	非零元远少于零元素的矩阵。三元组表 (行、列、值) 顺序存储，会失去随机存取特性；十字链表按行按列双向链接，适合矩阵运算后结构变化。

表2-3 串与数组名词

第3章 树与二叉树

树形结构引入「一对多」关系。主线：二叉树的性质与遍历是根基，遍历序列两两组合可唯一还原二叉树；线索化把遍历信息存进空指针；哈夫曼树解决最优编码；并查集与堆是两个工程化的树应用。B 树与 B+ 树属于查找章，见第 5 章。

3.1 树与二叉树基本性质

名词	英文/缩写	含义与要点
树	Tree	n ($n \geq 0$) 个结点的有限集： $n=0$ 为空树； $n>0$ 时有且仅一个根结点，其余结点分成 m ($m \geq 0$) 个互不相交的有限集，每个集又是一棵树（递归定义）。
结点的度 / 树的度	Degree	结点拥有的子树（孩子）数；树的度 = 各结点度的最大值。度 ≥ 3 的树没有对应二叉树的直接对应关系，但任何树都可经孩子兄弟表示法转为二叉树。
结点关系名词	—	根结点度数分层：根为第 1 层；深度（高度）为最大层数。孩子、双亲、兄弟、堂兄弟、祖先、子孙。
树的性质	—	结点数 = 总度数 + 1 (等于边数+1，因为除根外每结点对应一条入边)。度为 m 的树第 i 层至多 m^{i-1} 个结点； h 层 m 叉树至多 $(m^h - 1)/(m - 1)$ 个结点。
森林	Forest	m ($m \geq 0$) 棵互不相交的树的集合。删去树根即得森林。
二叉树	Binary Tree	每个结点至多两棵子树且子树有左右次序（有序树）。与度为 2 的树的区别：二叉树可为空、且左右子树次序不可颠倒。

名词	英文/缩写	含义与要点
满二叉树	Full / Perfect BT	每层结点都达最大值；同深度的二叉树中编号与满二叉树对应者为完全二叉树。注意英文文献口径差异，以中文教材定义为准。
完全二叉树	Complete BT	只允许最下层不满且最下层结点集中在左侧。性质： i 层满二叉树性质同样适用；编号 i 的结点，双亲 $\lfloor i/2 \rfloor$ ，左孩子 $2i$ ，右孩子 $2i+1$ ； n 个结点的完全二叉树高度 $\lceil \log_2(n+1) \rceil$ 。
二叉树重要性质	—	第 i 层至多 $2^{(i-1)}$ 个结点；深度 k 的二叉树至多 2^k-1 个结点；叶结点数 n_0 与度 2 结点数 n_2 满足 $n_0=n_2+1$ （由边数守恒推得）。
二叉树存储	—	顺序存储：按完全二叉树编号存数组，最坏（单支树）深度 k 需 2^k-1 单元，浪费严重；链式存储：二叉链表（data/lchild/rchild， n 个结点含 $n+1$ 个空指针域）、三叉链表增设 parent。

表3-1 树与二叉树名词

3.2 遍历与线索二叉树

名词	英文/缩写	含义与要点
先序 / 中序 / 后序遍历	Pre/In/Post-order	DLR / LDR / LRD（L 左 D 根 R 右），递归与非递归（栈实现）两种写法均需掌握，非递归中序最常考。 n 个结点共 $n+1$ 个空链域。
层序遍历	Level-order	借助队列，自上而下自左而右访问。
遍历序列还原	—	中序 + 先序（或中序 + 后序，或中序 + 层序）可唯一确定二叉树；先序 + 后序不能（无法区分左右子树边界），除非每个结点度数为 0 或 2。
线索二叉树	Threaded BT	利用 $n+1$ 个空指针存放遍历前驱后继：ltag=0 指左孩子、=1 指前驱；rtag=0 指右孩子、=1 指后继。中序线索最常用：找后继=右子树中最左下结点。引入线索后遍历无需栈和递归。

表3-2 遍历名词

3.3 树的存储、转换与哈夫曼树

名词	英文/缩写	含义与要点
树的三种存储	—	双亲表示法（数组+parent 指针，找双亲快找孩子慢）；孩子表示法（顺序+单链表拉链，找孩子快）；孩子兄弟表示法（左指针指向第一个孩子、右指针指向下一兄弟），本质是把树转成二叉树，是树/森林与二叉树转换的桥梁。
树↔二叉树转换	—	树：左孩子右兄弟加线抹线旋转即得二叉树，根无右子树；森林：每棵树各转二叉树后依次将后一棵树的根作为前一棵根的右孩子，转换后根有右子树。转换后的二叉树中序 = 原树的后序遍历次序（孩子兄弟视角）。
遍历对应关系	—	树的先根遍历 = 对应二叉树的先序遍历；树的后根遍历 = 对应二叉树的中序遍历；森林的先序和中序遍历与其二叉树表示的先序、中序遍历相同。

名词	英文/缩写	含义与要点
哈夫曼树 / 最优二叉树	Huffman Tree	n 个带权叶结点构成的二叉树中带权路径长度 $WPL=\sum w_i l_i$ 最小者。构造：每次取权值最小的两棵树合并，共 $n-1$ 次合并、总结点数 $2n-1$ ，只有度为 0 和 2 的结点。
哈夫曼编码	Huffman Coding	左分支 0 右分支 1（可互换），叶结点对应编码。前缀编码：任一编码都不是另一编码的前缀，保证译码唯一。总长最短；用于压缩。 n 个字符需 $n-1$ 次合并、编码树共 $2n-1$ 个结点。
堆	Heap	完全二叉树 + 父结点不小于（大根堆）或不大于（小根堆）孩子。建堆自底向上下滤 $O(n)$ ；插入/删除 $O(\log_2 n)$ 。是优先队列、堆排序、Top-K 问题的底层结构。
并查集	Disjoint Set	用双亲表示法存储的森林表示不相交集：Find 找根判断是否同集合，Union 合并两棵树。优化：按秩/大小合并 + 路径压缩，单次操作近似 $O(\alpha(n))$ ；Kruskal 求最小生成树依赖它判环。

表3-3 树与森林名词

第4章 图

图是「多对多」关系，名词密度最高的一章。主线：先选好存储结构（四种，各有适用场景），再谈遍历（BFS/DFS 是一切图算法的骨架），最后是五类经典算法：最小生成树两种、最短路径两种、拓扑排序与关键路径。

4.1 图的基本概念与存储

名词	英文/缩写	含义与要点
图 / 弧 / 边	Graph / Arc / Edge	顶点集 V 与边集 E 构成 $G=(V,E)$ ；有向图边为弧 $\langle x,y \rangle$ 有序，无向图为 (x,y) 无序对。不允许自环与重复边的简单图为考纲默认。
完全图 / 稀疏稠密	Complete Graph	无向完全图有 $n(n-1)/2$ 条边，有向完全图有 $n(n-1)$ 条弧。 $e < n \log_2 n$ 视为稀疏图（教材口径）。
度 / 入度 / 出度	Degree	顶点关联的边数；有向图分入度 ID 与出度 OD ， $TD=ID+OD$ 。握手定理：总度数 $=2 E $ 。有向图所有入度之和等于出度之和。
子图 / 生成子图	Subgraph	顶点和边均为原图子集；包含全部顶点的子图为生成子图。
连通 / 连通分量	Connected	无向图：顶点间有路径则连通；极大连通子图为连通分量。 n 个顶点连通图至少 $n-1$ 条边；边数少于 $n-1$ 必非连通。
强连通 / 强连通分量	Strongly Connected	有向图：任意两顶点互相可达。强连通有向图边数至少 n 。
生成树 / 生成森林	Spanning Tree	连通图的含全部 n 个顶点、仅 $n-1$ 条边的极小连通子图；加一条边成环，去一条边不连通。非连通图为生成森林。

名词	英文/缩写	含义与要点
邻接矩阵	Adjacency Matrix	$A[i][j]=1$ (或权值) 表示邻接, 无向图矩阵对称。特点: 空间 $O(n^2)$, 适合稠密图; 判断两顶点是否有边 $O(1)$; 求顶点度=行/列和; 遍历复杂度 $O(n^2)$ 。
邻接表	Adjacency List	每个顶点挂一条边链表。空间 $O(n+e)$, 适合稀疏图; 有向图求出度易求入度难 (需逆邻接表); 同一图的邻接表不唯一 (边链顺序可不同); 遍历复杂度 $O(n+e)$ 。
十字链表 / 邻接多重表	Orthogonal / Multilist	十字链表: 有向图的链式存储, 弧结点同时挂在弧尾链与弧头链上, 出入度都易求; 邻接多重表: 无向图的边只存一个结点, 每条边只有一个边结点, 适合对边操作的场合。

表4-1 图的名词与存储

4.2 遍历与经典算法

名词	英文/缩写	含义与要点
广度优先搜索	BFS	类似二叉树层序: 访问 v 后依次访问其未访问邻接点, 借助队列和 <code>visited[]</code> 标记数组。邻接表 $O(n+e)$ 、邻接矩阵 $O(n^2)$ 。按 BFS 可得 BFS 生成树, 非唯一 (取决于邻接次序)。可求无权图单源最短路径。
深度优先搜索	DFS	类似树的先序: 一条路走到黑再回溯, 借助栈/递归。复杂度同 BFS。DFS 生成树/森林; 有向图中可利用 DFS 检查强连通性 (逆向后序)。
最小生成树	MST	带权连通图边权和最小的生成树。MST 不一定唯一 (各边权互异时唯一), 但总权值唯一。
Prim 算法	—	从任一顶点出发, 每次选「树内顶点到树外顶点」的最小权边扩张。 $O(n^2)$, 与边数无关, 适合稠密图。
Kruskal 算法	—	按权值递增选不构成回路的边 (并查集判环), 选够 $n-1$ 条为止。 $O(e \log_2 e)$, 适合稀疏图。
Dijkstra 算法	—	单源最短路径, 贪心: 每轮确定一个最近顶点并松弛其邻边。 $O(n^2)$ (邻接矩阵); 不适用于带负权边的图。 <code>dist[]</code> <code>path[]</code> 更新过程是手算高频题。
Floyd 算法	—	各顶点间最短路径, 动态规划: 依次以 k 为中转尝试改进 $A[i][j]$ 。三重循环 $O(n^3)$; 允许有负权边但不允许负权回路。
拓扑排序	Topological Sort	AOV 网 (顶点=活动, 弧=先后关系) 中输出入度为 0 的顶点并删其出边, 重复直至全部输出; 若中途无零入度顶点则有环。 $O(n+e)$; 拓扑序列一般不唯一; 用 DFS 的逆后序也可得拓扑序列。DAG 有向无环图是前提。
关键路径	Critical Path	AOE 网 (顶点=事件, 弧=活动, 权=工期) 中最长路径, 决定工程最短工期。事件最早 ve (顺拓扑)、最迟 vl (逆拓扑); 活动最早开始 $e=ve$ (弧尾)、最迟 $l=vl$ (弧头)-权; $e=l$ 的活动为关键活动。缩短工期需压缩所有关键路径共有的关键活动。

表4-2 遍历与图算法名词

第5章 查找

查找章的主线是「有序性换效率」：从无序的顺序查找，到有序表的折半查找，再到把有序性维护进树（BST→AVL→红黑树→B/B+ 树），最后跳出树形用散列函数直接映射。每个结构都用 ASL（平均查找长度）评价。

5.1 线性与树形查找

名词	英文/缩写	含义与要点
查找表 / 关键字	Search Table / Key	由同一类型元素构成的集合；关键字是可标识元素的值，主关键字唯一。对查找表的操作：查询、检索、插入、删除；可插删的称动态查找表。
平均查找长度	ASL	$ASL = \sum P_i C_i$ (P_i 为查找第 i 个元素概率, C_i 为所需比较次数)。等概率下它是评价查找效率的核心指标，分成功与失败两种口径。
顺序查找	Sequential Search	从头扫到尾，哨兵（0 号位）可省越界判断。ASL 成功 $= (n+1)/2$ ，失败 $= n+1$ 。对顺序、链式、有序无序表都适用。
折半查找	Binary Search	仅适用于有序顺序表。 $mid = (low + high)/2$ ，比较后折半。判定树是一棵平衡二叉树； n 个元素最大比较次数 $= \lceil \log_2(n+1) \rceil$ ；成功 $ASL \approx \log_2(n+1) - 1$ 。元素奇偶、取整方向影响判定树形态，是手算重点。
分块查找	Blocking Search	分块有序：块间有序块内无序。先折半（或顺序）查索引表定块，再块内顺序查。ASL 介于顺序与折半之间；每块 $s = n/b$ 时 ASL 最小取 $\sqrt{n}$ 量级。
二叉排序树	BST	左子树所有结点 $<$ 根 $<$ 右子树所有结点，左右子树也各是 BST。中序遍历得递增序列。查找、插入 $O(h)$ ；删除分三种情况：叶直接删、单孩子接上、双孩子用中序前驱（或后继）替补。查找长度与树形有关，最坏（有序输入单支树）退化成 $O(n)$ 。
平衡二叉树	AVL Tree	任意结点左右子树高度差（平衡因子） $\in \{-1, 0, 1\}$ 的 BST。插入失衡后最小失衡子树旋转调整：LL→右旋、RR→左旋、LR→先左后右、RL→先右后左。删除可能引发向上多处调整。 n 层 AVL 最少结点数 $N_h = N_{h-1} + N_{h-2} + 1$ ，反之高度 $h \leq \log_2(n+1)$ 量级，查找 $O(\log_2 n)$ 。
红黑树	Red-Black Tree	每结点红或黑的 BST，五条性质：①结点红或黑 ②根黑 ③叶（NULL）黑 ④红结点孩子必黑（无连续红）⑤任一结点到其每个叶结点的黑高相等。保证最长路径 $\leq 2 \times$ 最短路径，高度 $\leq 2\log_2(n+1)$ ；插入 $O(\log_2 n)$ 且旋转至多 2 次。判「是不是红黑树」与插入染色是高频手算题。

表5-1 查找名词（一）

5.2 B 树、B+ 树与散列表

名词	英文/缩写	含义与要点
B 树	B-Tree	m 阶 B 树：多路平衡查找树。①每个结点至多 m 棵子树、m-1 个关键字 ②非根非叶至少 $\lceil m/2 \rceil$ 棵子树 ③根至少 2 棵子树（若无关键字可整体为空）④k 个关键字的结点有 k+1 棵子树 ⑤所有叶结点（失败结点）出现在同一层。插入溢出从中间分裂上移；删除不足 $\lceil m/2 \rceil - 1$ 先借后合。外部查找适合磁盘存取。
B+ 树	B+-Tree	m 阶 B+ 树：n 个关键字对应 n 棵子树；叶结点含全部关键字且用链表相连（支持顺序查找）；非叶结点只作索引（可重复出现关键字）。查找必到叶层。与 B 树的差异是常考对比题。
散列表	Hash Table	由散列函数 $H(\text{key})$ 直接建立关键字与存储地址的映射。冲突：不同关键字映射到同一地址，这些关键字互为同义词。散列查找效率与装填因子 $\alpha = \text{表中记录数} / \text{表长}$ 相关，与 n 本身无关（这是散列的本质优势）。
散列函数构造	—	除留余数法 $H(\text{key}) = \text{key} \% p$ （p 取不超过表长的最大素数）最常用；另有直接定址、数字分析、平方取中等。好的散列函数应均匀、冲突少。
开放定址法	Open Addressing	冲突后按探测序列找空位。线性探测 $H_i = (H + d_i) \% m$, $d_i = 1, 2, \dots$ ：易堆积（聚集）；平方探测 $d_i = 1^2, -1^2, 2^2, -2^2, \dots$ ：可缓解堆积，但探测序列必能覆盖的表长要求 $4k+3$ 型素数；双散列 $d_i = H_2(\text{key}) \cdot i$ 。注意：开放定址法下删除只能做惰性删除（打删除标记），否则截断探测链。
拉链法 / 链地址法	Chaining	同义词挂同一单链表。删除方便、装填因子可 > 1 ，但需额外指针空间。ASL 计算：成功 $= \sum \text{各链查找长度} / \text{元素数}$ ；失败 $= \sum \text{各链比较次数} / m$ 。
装填因子	Load Factor	$\alpha = \text{表中已存元素数} n / \text{散列表长度} m$ 。 α 越大冲突概率越大；散列表平均查找长度是 α 的函数。

表5-2 查找名词（二）

第6章 排序

排序是算法与数据结构综合运用最密集的一章。主线：五大家族（插入、交换、选择、归并、分配）各自的机制与性能，最后落到那张必背的排序对比总表（表 6-2），外部排序补充「内存放不下」的场景。

6.1 排序家族



图6-1 排序算法五大家族：判「族」看机制，判「优」看对比表

名词	英文/缩写	含义与要点
稳定性	Stability	排序后相等元素的相对次序保持不变的算法称稳定排序。稳定：直接插入、冒泡、归并、基数；不稳定：希尔、快排、简单选择、堆排。稳定性本身与正确性无关，是考点而非优点。
直接插入排序	Insertion Sort	把第 i 个元素插入前面已有序段。最好（有序） $O(n)$ 、最坏（逆序） $O(n^2)$ ，平均 $O(n^2)$ ，稳定，原地。基本有序或 n 小时是最佳选择。
折半插入排序	Binary Insertion	用折半查找确定插入位置：比较次数降为 $O(n\log_2 n)$ ，但移动次数不变，总复杂度仍 $O(n^2)$ ，稳定。
希尔排序	Shell Sort	按增量 d 分组做插入排序，增量逐趟缩小至 1。利用「基本有序后插入高效」；时间约 $O(n^{1.3})$ ，不稳定，仅适用于顺序存储。
冒泡排序	Bubble Sort	相邻逆序对两两交换，每趟把最大（小）元素冒到端部。某趟无交换可提前结束；最好 $O(n)$ ，平均/最坏 $O(n^2)$ ，稳定。
快速排序	Quick Sort	取枢轴 pivot 划分（partition）：左边 $\leq$ 枢轴 $\leq$ 右边，递归两边。平均 $O(n\log_2 n)$ （所有 $O(n\log_2 n)$ 排序中平均性能最优，常数最小）；最坏（基本有序/逆序取端点为枢轴） $O(n^2)$ ；空间为递归栈平均 $O(\log_2 n)$ 最坏 $O(n)$ ；不稳定。每趟至少将一个元素（枢轴）放到最终位置。优化：三者取中、小规模换插入排序。
简单选择排序	Selection Sort	每趟从未排序区选最小者与区首交换。比较次数固定 $n(n-1)/2$ ，与初始状态无关；移动次数至多 $3(n-1)$ ； $O(n^2)$ ，不稳定。
堆排序	Heap Sort	建大根堆后堆顶与堆尾交换、规模减 1、下滤重建，重复。建堆 $O(n)$ ，整体 $O(n\log_2 n)$ ；空间 $O(1)$ ；不稳定；只适用于顺序存储。适合求 Top-K 大规模数据。

名词	英文/缩写	含义与要点
归并排序	Merge Sort	两两归并有序段。2 路归并任何情况 $O(n\log_2n)$ ，稳定；需 $O(n)$ 辅助数组（空间是它相对快排的短板）； n 个元素需 $\lceil\log_2n\rceil$ 趟。
基数排序	Radix Sort	按位（最低位优先 LSD）依次分配-收集， r 个队列 d 趟。 $O(d(n+r))$ ， d 位数、 r 基数、 n 元素；稳定；不基于比较，适合位数少且 n 大的整数/字符串。
外部排序	External Sorting	内存装不下的文件排序：生成初始归并段（内部排序）+ 多趟 k 路归并。归并趟数 $S=\lceil\log_kr\rceil$ （ r 为初始段数）：增大 k 减少趟数但需败者树减少比较次数；减少 r 用置换-选择排序生成更长的初始段；最佳归并树（哈夫曼思想）安排归并顺序使 IO 最少；需增加 $k-1$ 个缓冲区避免内外存交互等待。

表6-1 排序算法名词

6.2 排序算法对比总表（必背）

算法	最好	平均	最坏	空间	稳定	备注
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是	基本有序时最优
折半插入	$O(n\log_2n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是	比较省、移动不减
希尔	$\approx O(n)$	$\approx O(n^{1.3})$	$O(n^2)$	$O(1)$	否	仅顺序存储
冒泡	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是	可提前终止
快排	$O(n\log_2n)$	$O(n\log_2n)$	$O(n^2)$	$O(\log_2n)$	否	平均最优；有序输入取端点最坏
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否	比较次数与初态无关
堆排	$O(n\log_2n)$	$O(n\log_2n)$	$O(n\log_2n)$	$O(1)$	否	建堆 $O(n)$
归并	$O(n\log_2n)$	$O(n\log_2n)$	$O(n\log_2n)$	$O(n)$	是	稳定且严格 $n\log_2n$
基数	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是	不基于比较

表6-2 九大排序性能对比（ n 元素， d 位数， r 基数）

第7章 跨章关系与易混辨析

数据结构的知识网可以用三条线收拢：逻辑结构—存储结构—算法复杂度（表 7-1）；树与图之间的结构类比（表 7-2）；以及一张易混名词辨析表（表 7-3）。答题前先问自己三个问题：数据怎么放？关系是什么？操作的量级是多少？

7.1 操作复杂度速查

结构	访问	查找	插入	删除	备注
顺序表	$O(1)$ 按位	$O(n)$ 按值	$O(n)$	$O(n)$	随机存取
单链表	$O(n)$	$O(n)$	$O(1)$ 已知前驱	$O(1)$ 已知前驱	顺序存取
栈 / 队列	—	—	$O(1)$ 端点	$O(1)$ 端点	受限访问
二叉排序树	—	$O(h)$	$O(h)$	$O(h)$	AVL 保证 $h=O(\log_2 n)$
散列表	—	$O(1)$ 期望	$O(1)$ 期望	$O(1)$ 期望	拉链法删除方便
B / B+ 树	—	$O(\log_2 n)$	$O(\log_2 n)$	$O(\log_2 n)$	磁盘友好

表7-1 核心结构核心操作复杂度 (n 元素)

7.2 易混名词辨析

易混组	一句话区分
满二叉树与完全二叉树	满=每层全满；完全=只许最下层缺且靠左
头指针与头结点	头指针必须有（访问入口）；头结点可有可无（统一操作）
循环队列判满 vs 判空	判空 $front == rear$ 恒成立；判满需牺牲单元/size/tag
树的度与二叉树的度	树的度看全树最大孩子数；二叉树度数只可能是 0/1/2 且分左右
先序+后序 与 其他组合	中序参与才唯一；先序+后序无法区分独子是左是右
BFS 与 DFS 的空间	BFS 借队列最坏 $O(n)$ ；DFS 借栈 $O(h)$ （树）/ $O(n)$ （链状图）
Prim 与 Kruskal	Prim 从点出发每次加点，稠密图；Kruskal 从边出发每次加边，稀疏图
Dijkstra 与 Floyd	单源贪心 $O(n^2)$ 忌负权；多源 DP $O(n^3)$ 可负权边不可负回路
拓扑与关键路径	拓扑保证先后；关键路径取最长，活动 $e=l$ 才关键
B 树与 B+ 树	B 树关键字子树相间查找可中途命中；B+ 树必到叶、叶链支持顺序遍历
散列 vs 树形查找	散列平均 $O(1)$ 但无序、需处理冲突；树形 $O(\log_2 n)$ 且支持范围/顺序
稳定排序的价值	仅在多关键字复合排序（先低后高再稳定排高位）时真正用得上

表7-2 高频易混对